SETUP DE ENTORNO

Plataforma Web de Invitaciones Digitales Personalizadas

Documento de Configuración y Despliegue del Entorno de Desarrollo y Producción

Versión:	1.0
Fecha:	11 de marzo de 2026
Stack:	React + NestJS + PostgreSQL + Firebase
MVP:	31 de marzo de 2026
Basado en:	SRS v1.0 | Modelo de Negocio v1.1 | Seguridad v1.0 | API REST v1.0 | BD v1.0

Documento confidencial — Solo para uso interno del equipo de desarrollo

1. Arquitectura General del Sistema

La plataforma sigue una arquitectura cliente-servidor con separación clara entre frontend y backend, alineada con la especificación del SRS v1.0 y el documento de Seguridad y Datos v1.0.

1.1 Diagrama de Componentes

Componente	Tecnología	Proveedor / Hosting
Frontend (SPA)	React + TypeScript	Vercel
Backend (API REST)	NestJS + TypeScript	Hostinger VPS
Base de Datos	PostgreSQL 16+	Hostinger VPS
Almacenamiento de Archivos	Firebase Storage	Google Cloud (Firebase)
Mapas	Google Maps (embebido)	Google Cloud
Mensajería	WhatsApp Web (wa.me/)	Sin integración API
Calendario	Google Calendar (link)	Google Cloud

1.2 Flujo de Comunicación

El frontend (Vercel) se comunica con el backend (Hostinger VPS) mediante API REST en formato JSON. El backend se conecta a PostgreSQL para datos relacionales y a Firebase Storage para archivos binarios (fotos, MP3). Todas las comunicaciones se realizan sobre HTTPS con CORS restringido exclusivamente a los dominios autorizados.

---

2. Requisitos Previos

2.1 Software Requerido

Software	Versión Mínima	Propósito	Notas
Node.js	v18 LTS o superior	Runtime para frontend y backend	Recomendado: v20 LTS
npm	v9+	Gestor de paquetes	Incluido con Node.js
PostgreSQL	16+	Base de datos relacional	Con extensión pgcrypto para UUID v4
Git	2.40+	Control de versiones	Commits claros, ramas organizadas
TypeScript	5.0+	Tipado estricto en frontend y backend	Instalado como dependencia del proyecto

Firebase CLI	Última estable	Gestión de Firebase Storage	npm install -g firebase-tools
Vercel CLI	Última estable	Deploy del frontend	npm install -g vercel

2.2 Cuentas y Servicios Requeridos

- **Vercel:** Cuenta para hosting del frontend React (plan gratuito suficiente para MVP).
- **Hostinger VPS:** Servidor virtual privado para backend NestJS y base de datos PostgreSQL.
- **Firebase (Google Cloud):** Proyecto configurado con Firebase Storage habilitado para fotos y archivos MP3.
- **Google Cloud Console:** API Key para Google Maps (embebido) y Google Calendar (link).
- **Dominio:** Dominio personalizado con certificado SSL/TLS. HTTPS obligatorio en todos los endpoints.
- **Servicio de Email:** Para notificaciones automáticas (nuevo pedido, expiración próxima, aviso de eliminación).

2.3 Presupuesto de Infraestructura

Según el Modelo de Negocio v1.1, el presupuesto máximo de infraestructura es de $80.000 ARS/mes, dentro de un presupuesto total de costos fijos de $100.000 ARS/mes.

---

3. Setup del Frontend (React + TypeScript)

3.1 Inicialización del Proyecto

```
# Crear proyecto React con TypeScript
npx create-react-app invitaciones-frontend --template typescript
cd invitaciones-frontend
```

3.2 Dependencias Principales

Paquete	Propósito
react-router-dom	Enrutamiento SPA: landing, invitación pública, galería, panel admin, etc.
axios / fetch	Comunicación con la API REST del backend (base URL: /v1).
@react-google-maps/api	Integración de Google Maps embebido en la página de invitación.
qrcode.react / similar	Generación de códigos QR para la galería de fotos.
react-countdown	Cuenta regresiva dinámica (días/h/m/s) actualizada cada segundo.
ESLint + Prettier	Linting y formateo de código según NFR-6.2 del SRS.

3.3 Variables de Entorno del Frontend

Crear archivo .env.local en la raíz del proyecto frontend:

```
REACT_APP_API_BASE_URL=https://api.invitaciones.com/v1
REACT_APP_GOOGLE_MAPS_API_KEY=<tu_api_key>
REACT_APP_FIREBASE_CONFIG=<configuracion_firebase_json>
REACT_APP_WHATSAPP_NUMBER=<numero_whatsapp_business>
```

3.4 Estructura de Rutas Principales

Ruta	Autenticación	Descripción
/	Pública	Landing page y catálogo
/:eventoId	Pública (URL UUID)	Página de invitación
/:eventoId?invitado=nombre	Pública	Invitación personalizada
/:eventoId/galeria	Pública (MVP)	Galería de fotos (QR)
/:eventoId/asistentes	Contraseña por evento	Lista de asistentes
/admin	JWT (usuario/contraseña)	Panel de administración

3.5 Consideraciones de Diseño

- Enfoque mobile-first responsive design (desde 320px hasta 4K).
- Compatibilidad: Chrome, Firefox, Safari, Edge (últimas 2 versiones).
- Accesibilidad WCAG 2.0 AA mínimo: contraste 4.5:1, navegación por teclado, etiquetas ARIA.
- Carga de landing page e invitaciones en menos de 5 segundos (P85).
- Lazy loading de imágenes y compresión JPEG/WebP.
- Minificación de CSS/JS en build de producción.
- Validación de formularios en frontend replicada obligatoriamente en backend (validación en capas).

3.6 Deploy en Vercel

```
# Instalar Vercel CLI
npm install -g vercel

# Deploy
vercel --prod
```

Configurar en el dashboard de Vercel las variables de entorno de producción y el dominio personalizado con SSL automático.

4. Setup del Backend (NestJS + TypeScript)

4.1 Inicialización del Proyecto

```
# Instalar Nest CLI
npm install -g @nestjs/cli

# Crear proyecto
nest new invitaciones-backend
cd invitaciones-backend
```

4.2 Dependencias Principales

Paquete	Propósito
@nestjs/typeorm + typeorm	ORM para PostgreSQL. Parámetros preparados para prevenir inyección SQL.
pg	Driver de PostgreSQL para Node.js.
@nestjs/jwt + passport-jwt	Autenticación JWT para panel de administración.
argon2 / bcrypt	Hashing de contraseñas con salting (Seguridad v1.0, sección 2.2).
class-validator + class-transformer	Validación automática de DTOs en todos los endpoints.
helmet	Cabeceras de seguridad: CSP, X-Content-Type-Options, X-Frame-Options, HSTS.
@nestjs/throttler	Rate limiting: 100 peticiones/minuto por IP.
firebase-admin	SDK para interactuar con Firebase Storage (fotos, MP3).
multer	Manejo de uploads multipart/form-data (fotos, música).
@nestjs/schedule	Cron jobs: eliminación automática de datos (02:00 AM) y limpieza de logs.
nodemailer / similar	Envío de notificaciones por email (nuevo pedido, expiración, eliminación).
sanitize-html	Sanitización HTML para prevenir XSS en campos de texto (Historia, etc.).

4.3 Variables de Entorno del Backend

Crear archivo .env en la raíz del proyecto backend:

```
# Servidor
PORT=3000
NODE_ENV=production

# Base de Datos
```

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=invitaciones_db
DB_USER=postgres
DB_PASSWORD=<contraseña_segura>

# JWT
JWT_SECRET=<secreto_largo_aleatorio>
JWT_EXPIRATION=24h

# Firebase
FIREBASE_PROJECT_ID=<tu_proyecto>
FIREBASE_STORAGE_BUCKET=<tu_bucket>.appspot.com
FIREBASE_SERVICE_ACCOUNT_PATH=./firebase-service-account.json

# CORS
CORS_ORIGINS=https://invitaciones.com,https://www.invitaciones.com

# Email
SMTP_HOST=<host_smtp>
SMTP_PORT=587
SMTP_USER=<email>
SMTP_PASS=<contraseña>
ADMIN_EMAIL=admin@invitaciones.com

# Rate Limiting
THROTTLE_TTL=60
THROTTLE_LIMIT=100

# Limites
MAX_PAYLOAD_SIZE=16mb
MAX_PHOTO_SIZE=15728640    # 15 MB en bytes
MAX_MUSIC_SIZE=20971520    # 20 MB en bytes
MAX_PHOTOS_PER_INVITATION=1000
```

4.4 Convenciones de la API REST

- **Base URL:** https://api.invitaciones.com/v1
- **Formato:** JSON (application/json) excepto uploads (multipart/form-data).
- **Versionado:** Prefijo /v1 en la URL base.
- **Nomenclatura:** Rutas en kebab-case, campos JSON en camelCase.
- **Timestamps:** ISO 8601 (YYYY-MM-DDTHH:mm:ssZ).
- **Payload máximo:** 16 MB por request.
- **Total de endpoints:** 48 (20 públicos, 27 con JWT, 1 con contraseña de evento).

4.5 Deploy en Hostinger VPS

Configurar el servidor VPS de Hostinger con Node.js, PostgreSQL y un proceso de gestión (PM2 recomendado):

```
# Instalar PM2 globalmente
npm install -g pm2

# Compilar TypeScript
npm run build

# Iniciar con PM2
pm2 start dist/main.js --name invitaciones-api
pm2 save
pm2 startup
```

Configurar Nginx como reverse proxy con SSL/TLS (Let's Encrypt) apuntando al puerto del backend.

---

5. Setup de Base de Datos (PostgreSQL 16+)

5.1 Creación de la Base de Datos

```
# Crear base de datos
sudo -u postgres psql
CREATE DATABASE invitaciones_db;
CREATE USER invitaciones_user WITH ENCRYPTED PASSWORD '<contraseña>';
GRANT ALL PRIVILEGES ON DATABASE invitaciones_db TO invitaciones_user;
\q
```

5.2 Extensiones Requeridas

```
-- Habilitar extensión para UUID v4
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Zona horaria por defecto
SET timezone = 'America/Argentina/Buenos_Aires';
```

5.3 Esquema de Tablas (14 tablas)

El esquema contiene 14 tablas organizadas en 4 grupos funcionales:

Grupo Funcional	Tablas
Catálogo y Configuración	Usuario, TipoEvento, Template, Servicio
Gestión Comercial	Pedido, PedidoServicio
Invitaciones y Contenido	Invitacion, InvitacionServicio, Invitado, HistoriaSeccion, FotoAnfitrion, Foto, Musica

Auditoría	Notificacion, LogEliminacion

La entidad central es Invitacion (UUID v4 como PK). Consultar el documento de Diseño de Base de Datos v1.0 para el script DDL completo y los datos semilla.

5.4 Convenciones de Nomenclatura

- **Tablas:** PascalCase en singular (Invitacion, TipoEvento, HistoriaSeccion).
- **Columnas:** snake_case (fecha_evento, tipo_evento_id, created_at).
- **Claves primarias:** id en todas las tablas. SERIAL para entidades estables, UUID para Invitacion.
- **Claves foráneas:** Formato {tabla_referenciada}_id (ej: invitacion_id, servicio_id).
- **Índices:** idx_{tabla}_{columna} para regulares, uq_{tabla}_{columna} para únicos.

5.5 Datos Semilla para MVP

El script de datos semilla incluye:

- 1 usuario administrador (contraseña hasheada con Argon2/bcrypt).
- 3 tipos de evento: Boda, Quinceañera, Cumpleaños.
- 9 servicios: 3 incluidos en base + 6 adicionales con precio en ARS.
- 9 templates: 3 por tipo de evento (placeholder para MVP).

5.6 Backups

```
# Backup completo semanal (cron: domingos 03:00 AM)
pg_dump -U postgres -Fc invitaciones_db \
  > /backups/invitaciones_$(date +%Y%m%d).dump

# Restauración
pg_restore -U postgres -d invitaciones_db \
  /backups/invitaciones_20260315.dump
```

Las confirmaciones de asistencia se clasifican como datos críticos. Se recomienda implementar WAL (Write-Ahead Logging) con archivado continuo para minimizar la ventana de pérdida de datos.

---

6. Setup de Firebase Storage

6.1 Configuración Inicial

- Crear proyecto en Firebase Console.
- Habilitar Firebase Storage en el proyecto.
- Generar archivo de credenciales de servicio (JSON) para el backend.
- Configurar reglas de almacenamiento en Firebase Console.

6.2 Límites de Almacenamiento

Recurso	Límite	Validación
Fotos de galería (por invitación)	Máx. 1.000 archivos	409 Conflict al exceder
Tamaño por foto	Máx. 15 MB	413 Payload Too Large
Archivo MP3	Máx. 20 MB	413 Payload Too Large
Fotos del anfitrión	Máx. 5 por invitación	409 Conflict al exceder
Texto de historia	Máx. 3.000 caracteres	400 Bad Request
Secciones de historia	Máx. 3 por invitación	409 Conflict al exceder

6.3 Verificación MIME

El backend valida que el contenido real del archivo coincida con la extensión declarada. No se confía en el nombre del archivo ni en el Content-Type del cliente. Extensiones permitidas para fotos: .jpg, .jpeg, .png, .webp, .gif. Para música: .mp3 (audio/mpeg).

6.4 Retención de Datos

Los archivos en Firebase Storage se retienen por 3 meses posteriores a la fecha del evento. El cron job nocturno (02:00 AM) elimina los archivos de Firebase Storage antes de ejecutar el hard delete en la base de datos. Los logs de eliminación se conservan por 6 meses.

---

7. Configuración de Seguridad

7.1 Autenticación por Niveles

Recurso	Mecanismo	Detalles
Panel Admin (/admin)	JWT + usuario/contraseña	Argon2/bcrypt. Bloqueo IP tras 5 intentos.
Lista de Asistentes	Contraseña por evento	Header X-Event-Password. Sin registro.
Invitaciones y Galería	Público (UUID v4)	Sin auth. URLs no predecibles.
Landing Page	Público	Sin autenticación requerida.

7.2 Cabeceras de Seguridad (Helmet)

Configurar Helmet en el backend NestJS para inyectar automáticamente:

- Content-Security-Policy (CSP)
- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY (prevención de clickjacking)
- Strict-Transport-Security (HSTS)

7.3 CORS

CORS restringido exclusivamente a los dominios del frontend (Vercel) y panel admin. Configurar en el backend:

```
app.enableCors({
  origin: ['https://invitaciones.com', 'https://www.invitaciones.com'],
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  credentials: true,
});
```

7.4 Rate Limiting

Máximo 100 peticiones por minuto por IP. Header X-RateLimit-Remaining en cada respuesta. Bloqueo temporal de IP tras 5 intentos fallidos de login (protección contra fuerza bruta).

7.5 Protección contra Ataques

Amenaza	Medida	Capa
Inyección SQL	TypeORM con parámetros preparados. Sin strings crudos.	Backend
XSS	Sanitización HTML + CSP headers.	Backend / Frontend
Fuerza bruta	Bloqueo temporal de IP tras 5 intentos fallidos.	Backend
DoS	Rate limiting: 100 req/min por IP.	Backend
Subida de malware	Restricción de extensiones + verificación MIME + límite 15 MB.	Backend / Firebase
Buffer overflow	Payload máximo de 16 MB por request.	Backend
Clickjacking	X-Frame-Options + CSP via Helmet.	Backend

7.6 Normativas

- **Ley 25.326 (Argentina):** Consentimiento explícito para datos personales. Derechos de acceso, rectificación y supresión.
- **GDPR:** Aplicable si usuarios de la UE acceden a la plataforma.
- **Documentos legales requeridos:** Términos de Servicio, Política de Privacidad, Aviso de cookies, Disclaimers.

---

8. Integraciones Externas

8.1 Google Maps

- API Key configurada en Google Cloud Console con restricciones de dominio.
- Uso: mapa embebido en la página de invitación con pin del evento.

- Botón “Abrir en Google Maps” que abre en app/web nativa.
- Fallback: dirección texto + link manual en caso de falla de la API.

8.2 Google Calendar

Integración mediante link estándar de Google Calendar. No requiere API Key adicional. Botón “Agregar a mi calendario” en la invitación.

8.3 WhatsApp Business

Sin integración API. Se utiliza el formato wa.me/<número> para generar links directos. El contacto con el anfitrión se realiza por WhatsApp Business.

8.4 Integraciones NO incluidas en MVP

- Stripe, PayPal, MercadoPago (pagos fuera de la plataforma).
- Email marketing APIs.
- Redes sociales.

---

9. Automatizaciones y Cron Jobs

9.1 Cron Job: Eliminación Automática de Datos

Parámetro	Valor
Frecuencia	Cada noche a las 02:00 AM (America/Argentina/Buenos_Aires)
Condición	Invitaciones cuyo fecha_expiracion < CURRENT_DATE AND activa = TRUE
Acciones	1) Registrar LogEliminacion. 2) Eliminar archivos de Firebase Storage. 3) Hard delete en BD (CASCADE).

9.2 Cron Job: Limpieza de Logs

Cron job mensual que elimina registros de LogEliminacion con más de 6 meses de antigüedad.

9.3 Notificaciones Automáticas por Email

Tipo	Destinatario	Trigger
NUEVO_PEDIDO	Propietario	Anfitrión completa formulario en landing page.
EXPIRACION_PROXIMA	Anfitrión	7 días y 3 días antes de eliminación.
AVISO_ELIMINACION	Interno (log)	Cron job ejecuta eliminación de datos.

---

10. Calidad de Código y Testing

10.1 Estándares de Código

- TypeScript con tipos estrictos en frontend y backend.
- Linting: ESLint + Prettier con configuración compartida.
- Git: commits claros y descriptivos, ramas bien organizadas.
- Documentación de código, API (endpoints, parámetros, respuestas) y BD (esquema, relaciones).

10.2 Testing

- Cobertura mínima de código: 60% (NFR-6.2 del SRS).
- Tests unitarios para lógica de negocio (cálculo de precios, validaciones).
- Tests de integración para endpoints de la API.
- Validar tiempos de carga: landing page e invitaciones < 5 segundos (P85).
- Validar tiempo de respuesta de API < 500ms (media).

10.3 Métricas de Rendimiento

Métrica	Objetivo
Carga de landing page	< 5 segundos (P85)
Carga de invitaciones	< 5 segundos (P85)
API endpoints (media)	< 500 ms
Cálculo de precio dinámico	< 100 ms
Confirmación de asistencia	< 2 segundos
Upload de foto (15 MB)	< 5 segundos
Concurrencia máxima	20 invitaciones simultáneas
Uptime SLA	99.5% anual (máx. 3.6h downtime/año)

---

11. Checklist de Setup Completo

Utilizar este checklist para verificar que todos los componentes estén configurados antes del despliegue:

Entorno de Desarrollo

- Node.js v18+ y npm v9+ instalados.
- PostgreSQL 16+ instalado y ejecutándose con extensión pgcrypto.
- Git configurado con credenciales.
- Firebase CLI y Vercel CLI instalados globalmente.

Frontend

- Proyecto React + TypeScript creado e inicializado.
- Dependencias instaladas (router, maps, QR, countdown, etc.).
- Variables de entorno (.env.local) configuradas.
- ESLint + Prettier configurados.
- Build de producción exitoso (npm run build).

Backend

- Proyecto NestJS creado e inicializado.
- Dependencias instaladas (TypeORM, JWT, Helmet, Throttler, Firebase Admin, etc.).
- Variables de entorno (.env) configuradas.
- Archivo firebase-service-account.json ubicado y referenciado.
- Helmet y CORS configurados.
- Rate limiting habilitado (100 req/min por IP).
- Cron jobs configurados (eliminación nocturna, limpieza de logs).
- Build de producción exitoso (npm run build).

Base de Datos

- Base de datos invitaciones_db creada.
- Extensión pgcrypto habilitada.
- Script DDL ejecutado (14 tablas + índices).
- Datos semilla insertados (admin, tipos de evento, servicios, templates).
- Backup automatizado configurado (semanal, domingos 03:00 AM).

Firebase Storage

- Proyecto Firebase creado.
- Firebase Storage habilitado.
- Reglas de almacenamiento configuradas.
- Credenciales de servicio descargadas y configuradas en el backend.

Seguridad

- HTTPS configurado en todos los endpoints (SSL/TLS).
- Cabeceras de seguridad (Helmet) activas.
- CORS restringido a dominios autorizados.
- Contraseña de admin hasheada con Argon2/bcrypt.
- Bloqueo de IP tras 5 intentos fallidos de login.

Producción

- Frontend desplegado en Vercel con dominio personalizado.
- Backend desplegado en Hostinger VPS con PM2 y Nginx.
- Dominio y certificados SSL configurados.
- Rollback automático configurado en caso de error crítico.
- Página de mantenimiento estática lista para contingencias.
- Documentos legales publicados: TOS, Política de Privacidad, Cookies, Disclaimers.

Fin del Documento — Setup de Entorno v1.0